Computer Programming 1

Functions (part 1)

(28-09-2026)

Outline

- Introduction (Functions, Library of functions, Procedures, …)
- Definition, Declaration and Call of Functions
- Function parameters and local variables
- Parameters passing
- Recursive functions

Outline

- Introduction (Functions, Library of functions, Procedures, …)
- Definition, Declaration and Call of Functions
- Function parameters and local variables
- Parameters passing
- Recursive functions

Outline

- **Introduction (Functions, Library of functions, Procedures, …)**
- Definition, Declaration and Call of Functions
- Function parameters and local variables
- Parameters passing
- Recursive functions

Introduction to the function/s

In a program, it is convenient to structure the code by grouping its parts in autonomous sub-programs, called functions, which are executable in each point where it is required.

A function can be used repeatedly in a program, or in different programs

- The organization in functions has many advantages:
  - Better structuring and organization of the code
  - Increase readability of the code
  - Increase maintainability of the code
  - Re-use of sub-parts of the same program several times
  - Using code made by others / libraries
  - ...
- In a C ++ program
  - it is possible to define and call functions
  - It is also possible to call functions defined elsewhere
    - functions defined in other files
    - library functions

Library functions

A library is a set of precompiled functions.

- Some C ++ (standard) libraries are available in all implementations and with the same functions (e.g., iostream, cmath)
- A library consists of a pair of files:
  - a header file containing the declarations of the sub-programs themselves
  - an object file containing the compiled functions
- To use the functions in a library in a program you must:
  - include the library header file with the #include <libraryname> directive
  - in some cases, indicate to the linker the file containing the compiled functions of the library
    - E.g., #include "myLibraryName.h"
  - The #include directive introduces in the program calls to the functions of the library

Typically, this form refers to the standard libraries or libraries of OS

Typically, this form refers to the local libraries

List of standard libraries: https://en.cppreference.com/w/cpp/header

Some examples of library functions (1)

Library <cstdlib>: it provides miscellaneous utilities

- rand(): it returns a pseudo-random integral value from the range [0, RAND_MAX].
  - RAND_MAX is the max of int numbers
  - **e.g.,** rand() % N: **it returns a number between 0 and N-1**
  - **e.g.,** rand() % (max - min + 1) + min: **it returns a number in the range** [min, max]
  - e.g., min + rand() % (max - min + 1) : **it returns a number in the range [-min, +max]**

Example with rand (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include<cstdlib>
4.  int main()
5.  {
6.   int v0= rand();     // [0, RAND_MAX]
7.   cout << "(v0) " << v0 << " RAND_MAX=" << RAND_MAX << endl;
8.   int v1 = rand() % 100;        // [0, 99]
9.   cout << "(v1) " << v1 << endl;
10.  int v2= rand() % 100; // [0, 99]
11.  cout << "(v2) " << v2 << endl;
12.  int v3 = rand() % (100-10+1) + 10; // [10, 100]
13.  cout << "(v3) " << v3 << endl;
14.  return 0;
15. }
```

first execution

```
$ g++ random.cc
$ ./a.out
(v0) 1804289383 RAND_MAX=2147483647
(v1)  86
(v2)  94
(v3)  26
```

Example:

{ ../FUNCTIONS/random.cc}

Example with rand (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include<cstdlib>
4.  int main()
5.  {
6.   int v0= rand();     // [0, RAND_MAX]
7.   cout << "(v0) " << v0 << " RAND_MAX=" << RAND_MAX << endl;
8.   int v1 = rand() % 100;        // [0, 99]
9.   cout << "(v1) " << v1 << endl;
10.  int v2= rand() % 100; // [0, 99]
11.  cout << "(v2) " << v2 << endl;
12.  int v3 = rand() % (100-10+1) + 10; // [10, 100]
13.  cout << "(v3) " << v3 << endl;
14.  return 0;
15. }
```

first execution

```
$ g++ random.cc
$ ./a.out
(v0) 1804289383 RAND_MAX=2147483647
(v1)  86
(v2)  94
(v3)  26
```

second execution

```
$ ./a.out
(v0) 1804289383 RAND_MAX=2147483647
(v1)  86
(v2)  94
(v3)  26
```

Example:
{ ../FUNCTIONS/random.cc}

Some examples of library functions (2)

Library <cstdlib>: it provides miscellaneous utilities

- rand(): It returns a pseudo-random integral value from the range [0, RAND_MAX].
  - RAND_MAX is the max of int numbers
  - e.g., rand() % N: it returns a number between 0 and N-1
  - e.g., rand() % (max - min + 1) + min: it returns a number in the range [min, max]
  - e.g., min + rand() % (max - min + 1) : it returns a number in the range [-min, +max]
- srand(x): It seeds the pseudo-random number generator used by rand() with the value seed. To be used before rand().
  - srand(time(nullptr)), srand(time(NULL)), or srand(time(0)) use the current time as seed
  - each time rand() is seeded with the same seed, it must produce the same sequence of values
  - without using srand() the random generator rand() uses the default value, i.e., srand(1)

Library <ctime>: it provides time-related utilities

- time(nullptr): it returns the current calendar time encoded as a time_t object

https://cplusplus.com/reference/cstdlib

Example with rand (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include<cstdlib>
4.  #include<ctime>
5.  int main()
6.  {
7.   srand(time(NULL));
8.   int v0= rand();     // [0, RAND_MAX]
9.   cout << "(v0) " << v0 << " RAND_MAX=" << RAND_MAX << endl;
10.  int v1 = rand() % 100;        // [0, 99]
11.  cout << "(v1) " << v1 << endl;
12.  int v2= rand() % 100; // [0, 99]
13.  cout << "(v2) " << v2 << endl;
14.  int v3 = rand() % (100-10+1) + 10; // [10, 100]
15.  cout << "(v3) " << v3 << endl;
16.  return 0;
17. }
```

first execution

```
$ g++ random.cc
$ ./a.out
(v0) 1804289383 RAND_MAX=2147483647
(v1)  86
(v2)  94
(v3)  26
```

second execution

```
$ g++ random.cc
$ ./a.out
(v0) 754211445 RAND_MAX=2147483647
(v1)  9
(v2)  84
(v3)  36
```

Example:

{ ../FUNCTIONS/random_2.cc}

Some examples of library functions (2)

Library <cmath>: mathematical functions (from double to double)

- fabs (x): absolute value of type float (real numbers)
- abs (x): absolute value of type integer (it can be used for integers and also for real numbers)
- sqrt (x): square root of x
- pow (x, y): it raises x to the power of y
- exp (x): exponential value of x
- log (x): natural logarithm of x
- log10 (x): base-10 logarithm of x
- sin (x) and asin (x): trigonometric sine and arcsine
- cos (x) and acos (x): cosine and arccosine trigonometric
- tan (x) and atan (x): tangent and arctangent trig
- round (x): returns the integral value that is nearest to x (with halfway cases rounded away from zero)
- trunc (x): it returns the integer value of x (rounds x toward zero)
- …

These functions can be used with all other numeric types via implicit / explicit conversion

https://cplusplus.com/reference/cmath

Some examples of library functions (3)

Library <cctype>, recognition functions (from char to bool):

- isalnum (c): alphabetic character or decimal digit
- isalpha (c): alphabetic character
- iscntrl (c): control character (e.g., in ASCII table all characters between 0x00 and 0x1f)
- isdigit (c): decimal place (e.g., any of 0 1 .. 9)
- isupper (c): capital letter
- islower (c): lowercase letter
- isprint (c): printable character, also space
- isspace (c): it returns 0 if c is not a space, a page break, new line or tab, otherwise a non-zero value is returned
- isxdigit (c): hexadecimal digit

Library <cctype>, conversion functions (from char to char):

- tolower (c): if c is an uppercase letter, it returns the corresponding letter lowercase, otherwise it returns c
- toupper (c): as above but capitalized
- …

https://cplusplus.com/reference/cctype

Example:

{ ../FUNCTIONS/tolowercase.cc}

Example with tolowercase (3)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cctype>
4.  int main()
5.  {
6.   char cu='A';
7.   char cl=tolower(cu);
8.   cout << "cu= " << cu << " cl=" << cl << " (int)cu" << (int)cu << endl;
9.   cout << endl;
10.  int indc=65;
11.  cl=tolower(indc);
12.  cout << "indc=" << indc;
13.  cout << " cu=" << (char) indc<< " cl=" << cl << endl;
14.  cout << endl;
15.  cout << isspace((char)indc) << endl;
16.  cout << isspace((char)32) << endl;
17.  cout << isspace(' ') << endl;
18.  return 0;
19. }
```

```
$ g++ tolowercase.cc
$ ./a.out
cu=A cl=a (int)cu=65


indc= 65 cu=A cl=a


0
8192
8192
```

Example:

{ ../FUNCTIONS/tolowercase.cc}

ASCII table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

https://www.asciitable.com

Some examples of library functions (4)

Library <climits>, it defines constants with the limits of fundamental integral types for the specific system and compiler implementation used (i.e., sizes of integral types)

- INT_MAX
- INT_MIN

..

Library <cfloat>, it describes the characteristics of floating types for the specific system and compiler implementation used.

- FLT_MAX
- DBL_MAX
- LDBL_MAX
- FLT_MIN
- DBL_MIN
- LDBL_MIN
- FLT_EPSILON
- DBL_EPSILON
- LDBL_EPSILON

..

No functions, but constant values

https://cplusplus.com/reference/climits
https://cplusplus.com/reference/cfloat

Some examples of library functions (APIs)

Library <cstdlib>

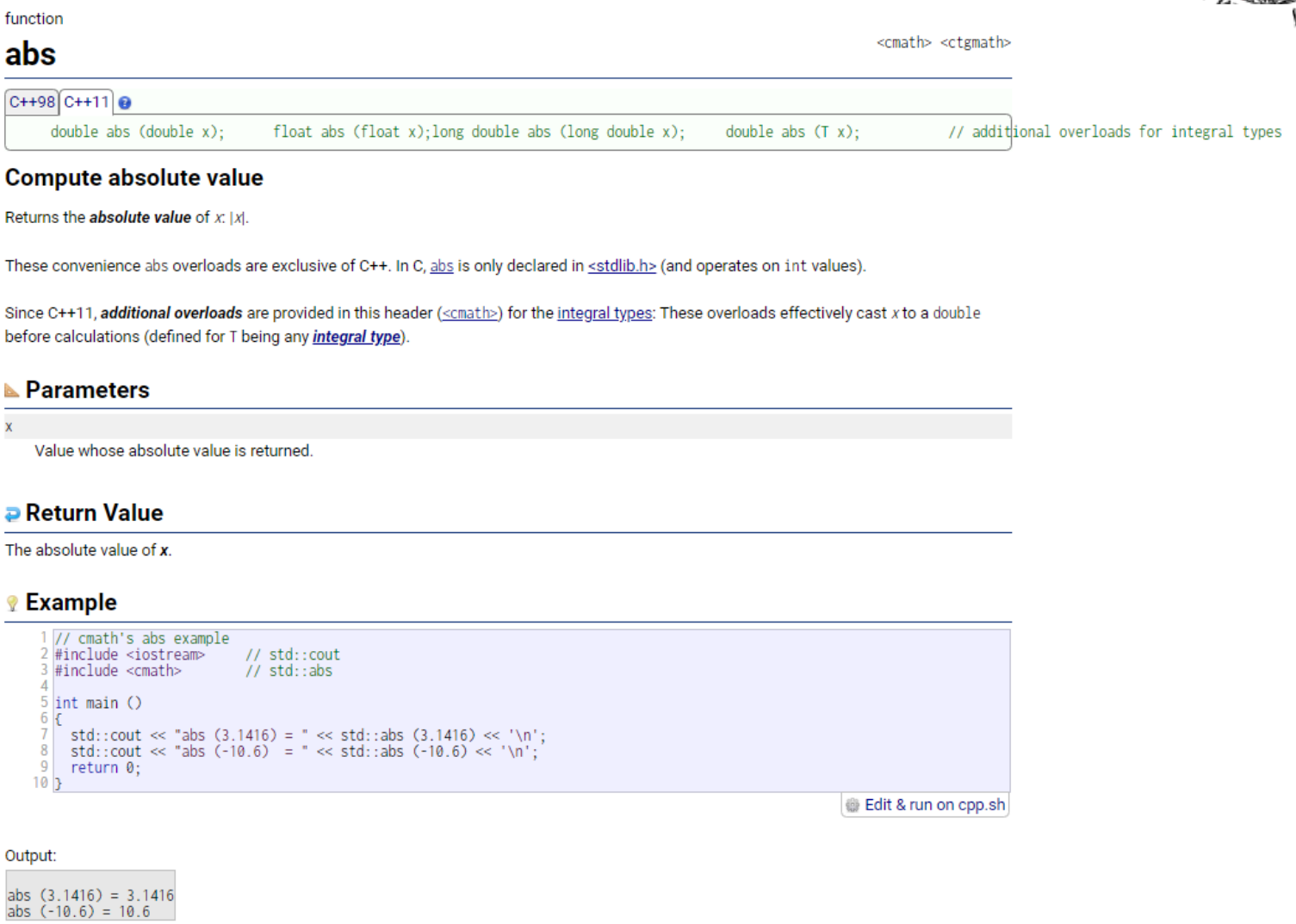
Library <cctype>

Library <cmath>

Library <climits>

Library <cfloat>





https://cplusplus.com/reference/cstdlib
https://cplusplus.com/reference/cmath
https://cplusplus.com/reference/cctype
https://cplusplus.com/reference/climits
https://cplusplus.com/reference/cfloat

Example of execution flow of a library function

Example

```
#include <cmath>

1. int main(){
2. (...)
3.    for (float index=1.0; index<=MAX; index+=1.0)
4.          cout << log(index) / log(2.0) << endl;          // function call
5. (...)
6. }
```

```
// from the cmath library header :
    double log(double x);
```

- Execution flow of the function call log(index):
  1. the value of index is evaluated and passed as input to the function log
  2. the program transfers the control from the code of main function to the code of log function in cmath, and it resumes at the end of the function
  3. The value returned by the computation of the log function with the index value in input is used as log(index)

Example above (extended):

{ ../FUNCTIONS/table_logarithms.cc} { ../FUNCTIONS/precision_1.cc, precision.cc}

Outline

- Introduction (Functions, Library of functions, Procedures, …)
- **Definition, Declaration and Call of Functions**
- Function parameters and local variables
- Parameters passing
- Recursive functions

Functions: declarations, definition and call

- Definition
  - Syntax: type id(type1 id1, … typeN idN) { … }
  - Id1, …, idN are the formal parameters of the function
  - E.g., double pow(double x, double y) { … }
- Declaration
  - Syntax: type id(type1 id1, … typeN idN);
  - id1, …, idN are optional parameters
  - E.g., double pow(double, double e);
  - It is used to "recall" a definition made elsewhere and allow its use!
- Call
  - Syntax: id (exp1, …, expN)
  - exp1, … , expN are the actual/current parameters of the call
  - E.g., x = pow(2.0*y, 3.0)

Example

```
// definition
        int gcd(int a, int b){
          // body of the function…..
         return 0;
        }
// declaration
        int gcd(int, int);
// call
        gcd(24,36);
        int ret = gcd(24,36);
```

Note

- The actual parameters exp1, ..., expN of the call must be compatible for number, order and respectively by type to the corresponding formal parameters!
- The ***signature of a function*** is composed of: (i) the function name; (ii) number, order and type of formal parameters
  - Note that the type of the return parameter is not part of the signature

Examples of code structure (1)

```
1.  using namespace std;
2.  #include <iostream>


3.  int gcd(int a, int b){
4.      return 12;
5.  }


6.  int main()
7.  {
8.      int res=gcd(24,36);
9.      cout << res << endl;
10. }
```

Examples of code structure (2)

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int gcd(int a, int b){
5.      return 12;
6.  }
7.
8.  int main()
9.  {
10.     int res=gcd(24,36);
11.     cout << res << endl;
12. }
```

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int main()
5.  {
6.      int res=gcd(24,36);
7.      cout << res << endl;
8.  }
9.
10. int gcd(int a, int b){
11.     return 12;
12. }
```

Examples of code structure (2)

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int gcd(int a, int b){
5.      return 12;
6.  }
7.
8.  int main()
9.  {
10.     int res=gcd(24,36);
11.     cout << res << endl;
12. }
```

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int main()
5.  {
6.      int res=gcd(24,36);
7.      cout << res << endl;
8.  }
9.
10. int gcd(int a, int b){
11.     return 12;
12. }
```

Pay attention to the order of the function definitions

Examples of code structure (2)

```
1.  using namespace std;
2.  #include <iostream>

3.  int gcd(int a, int b){
4.      return 12;
5.  }

6.  int main()
7.  {
8.      int res=gcd(24,36);
9.      cout << res << endl;
10. }
```

```
1.  using namespace std;
2.  #include <iostream>

3.  int gcd(int, int);  //function declaration

4.  int main()
5.  {
6.      int res= gcd(24,36);
7.       cout << res << endl;
8.  }

9.  int gcd(int a, int b){
10.     return 12;
11. }
```

The instruction *return*

- The body of a function can contain one or more return instructions:
  Syntax: return exp
  - epx must be compatible with the type returned from the function
  - E.g., return 3*x;

- The execution of the return instruction
  - It ends the function
  - It causes the value of the function call to be the value of the expression exp (with implicit conversion if of a different type)

Note

- It is a good practice that a function contains only one instruction return

Example about the use of a function

Consider the call *int res=gcd(n1, n2)*

Execution steps:

i. the current parameter values n1 and n2 are calculated (the order is not specified)

ii. the values are copied, in order, in the formal parameters a and b (*call by value*)

iii. the function is executed and the values of a, b, and of the remainder local variable are modified (n1 and n2 remain with their original value)

iv. the gcd function returns to the calling program the value of the expression that appears in the return statement (the value is copied in res)

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example of function

```
int gcd(int a, int b){
   int remainder;
   while(b!=0) {
      remainder = a%b;
      a = b;
      b = remainder;
   }
   return a;
}
```

Example of function

{ ../FUNCTIONS/gcd.cc, gcf.cc}

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-						

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-						

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24				

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef
4	-	24	36	24	36	24

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef
4	-	24	36	24	36	24
5	-	24	36	36	36	24

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef
4	-	24	36	24	36	24
5	-	24	36	36	36	24
6	-	24	36	36	24	24

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef
4	-	24	36	24	36	24
5	-	24	36	36	36	24
6	-	24	36	36	24	24
3	-	24	36	36	24	24
4	-	24	36	36	24	12
5	-	24	36	24	24	12
6	-	24	36	24	12	12

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef
4	-	24	36	24	36	24
5	-	24	36	36	36	24
6	-	24	36	36	24	24
3	-	24	36	36	24	24
4	-	24	36	36	24	12
5	-	24	36	24	24	12
6	-	24	36	24	12	12
3	-	24	36	24	12	12
4	-	24	36	24	12	0
5	-	24	36	12	12	0
6	-	24	36	12	0	0

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef
4	-	24	36	24	36	24
5	-	24	36	36	36	24
6	-	24	36	36	24	24
3	-	24	36	36	24	24
4	-	24	36	36	24	12
5	-	24	36	24	24	12
6	-	24	36	24	12	12
3	-	24	36	24	12	12
4	-	24	36	24	12	0
5	-	24	36	12	12	0
6	-	24	36	12	0	0
3	-	24	36	12	0	0

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef
4	-	24	36	24	36	24
5	-	24	36	36	36	24
6	-	24	36	36	24	24
3	-	24	36	36	24	24
4	-	24	36	36	24	12
5	-	24	36	24	24	12
6	-	24	36	24	12	12
3	-	24	36	24	12	12
4	-	24	36	24	12	0
5	-	24	36	12	12	0
6	-	24	36	12	0	0
3	-	24	36	12	0	0
8	-	24	36	12	0	0

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Example about the use of a function: example

```
n1=24;
n2=36;
int res=gcd(n1, n2);
```

Example of function

```
1.  int gcd(int a, int b){
2.      int remainder;
3.      while(b!=0) {
4.          remainder = a%b;
5.          a = b;
6.          b = remainder;
7.      }
8.      return a;
9.  }
```

Modulus operator (recall)
3%2=1 rem.1
4%2=2 rem.0

line	res	n1	n2	a	b	reminder
-	-	24	36			
1	-	24	36	24	36	
2	-	24	36	24	36	undef
3	-	24	36	24	36	undef
4	-	24	36	24	36	24
5	-	24	36	36	36	24
6	-	24	36	36	24	24
3	-	24	36	36	24	24
4	-	24	36	36	24	12
5	-	24	36	24	24	12
6	-	24	36	24	12	12
3	-	24	36	24	12	12
4	-	24	36	24	12	0
5	-	24	36	12	12	0
6	-	24	36	12	0	0
3	-	24	36	12	0	0
8	-	24	36	12	0	0
-	12	24	36			

gcd(n1,n2) is the greatest common divisor (GCD) between two numbers n1 and n2, e.g., gcd(24,36)=12

Examples

Example of mixed calls to defined functions and libraries:

{ ../FUNCTIONS/mylog10.cc}

Example of factorial:

{ ../FUNCTIONS/fact.cc}

Example of declaration (header) :

{ ../FUNCTIONS/fact1.cc}

Example with local and global identifier “factorial”:

{ ../FUNCTIONS/fact2.cc}

Example with formal parameter having the same name of an actual parameter:

{ ../FUNCTIONS/fact3.cc}

Example of decomposition in more files:

{ ../FUNCTIONS/fact4*.{cc|h}

Example of Boolean function:

{ ../FUNCTIONS/isprime.cc}

Examples

Example of mixed calls to defined functions and libraries:

{ ../FUNCTIONS/mylog10.cc}

Example of factorial:

{ ../FUNCTIONS/fact.cc}

Example of declaration (header) :

{ ../FUNCTIONS/fact1.cc}

Example with local and global identifier “factorial”:

{ ../FUNCTIONS/fact2.cc}

Example with formal parameter having the same name of an actual parameter:

{ ../FUNCTIONS/fact3.cc}

Example of decomposition in more files:

{ ../FUNCTIONS/fact4*.{cc|h}

Example of Boolean function:

{ ../FUNCTIONS/isprime.cc}

Task for understanding the code

1. Compile and run
2. Observe the result
3. Understand why the result is produced
   - by observing the code
   - By simulating the execution
4. Try to change the code and observe the changes in the results

Procedure (*void* functions)

Example

```
void funct (int x){...}   //definition of void function (no return or "return;")
(...)
funct(n*2);               // call of the void function
```

- Functions that return void are named **Procedure**
- In a function with return type void, the return statement can be missing, or the return expression can be omitted (e.g., "return;")

Example of void function, print of date:

{ ../FUNCTIONS/printdate.cc}

Example of void function, print of all chars:

{ ../FUNCTIONS/printchartype.cc}

Example of void function without arguments:

{ ../FUNCTIONS/rolldice.cc}

{ ../FUNCTIONS/random_number.cc}

Multiple *return* in a function: example

- In a function, the use of multiple return statements has to be avoided, especially in *if-then-else* blocks

```
int f(int a, int b){
  if (a>b)
   return 1;
  else
   return 0;
}
```



```
int f(int a, int b){
   int res=0;
   if (a>b)
      res = 1;
   else
      res = 0;
 return res;
}
```

Example of isprime function with unique return:

{ ../FUNCTIONS/isprime_onereturn.cc}

The instruction *return* in a loop (implicit jump)

- The return statement ends directly both cycle and function
  - It must be avoided →it can be done by using conditions

```
1.  int f(…){
2.  …
3.    while (…) {
4.          ….
5.          return …;
6.          …
7.     }
8.  …
9.  }
```

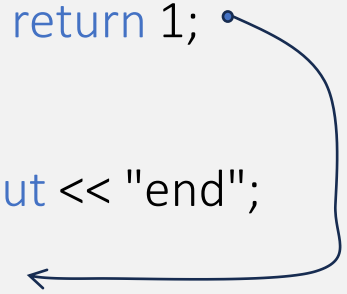
The flow: from (5) to (8)

The instruction *return* in a loop (implicit jump)

- The return statement ends directly both cycle and function
  - It must be avoided →it can be done by using conditions
  - It is different from break;

Example with return

```
1. int f(int a, int b){
2. while (a>b){
3.     a -= 1;
4.     if (a<0)
5.         return 1;
6.    }
7.   cout << "end";
8. }
```



The instruction *return* in a loop (implicit jump)

- The return statement ends directly both cycle and function
  - It must be avoided →it can be done by using conditions
  - It is different from break

Example with return

```
1. int f(int a, int b){
2. while (a>b){
3.     a -= 1;
4.     if (a<0)
5.         return 1;
6.   }
7.   cout << "end";
8. }
```


Example with break

```
1. int f(int a, int b){
2. while (a>b){
3.     a -= 1;
4.     if (a<0)
5.         break;
6.   }
7.   cout << "end";
8. }
```



Implicit conversion of actual parameters

- The call (for value) is logically analog at the initialization of the formal parameters

```
int f( int x ...){ … }        //function definition

....

.... f(exp) …                 //function call
```

int x = exp;

exp is evaluated and then assigned to the parameter x

```
int gcd( int a, int b){ … }   //function definition

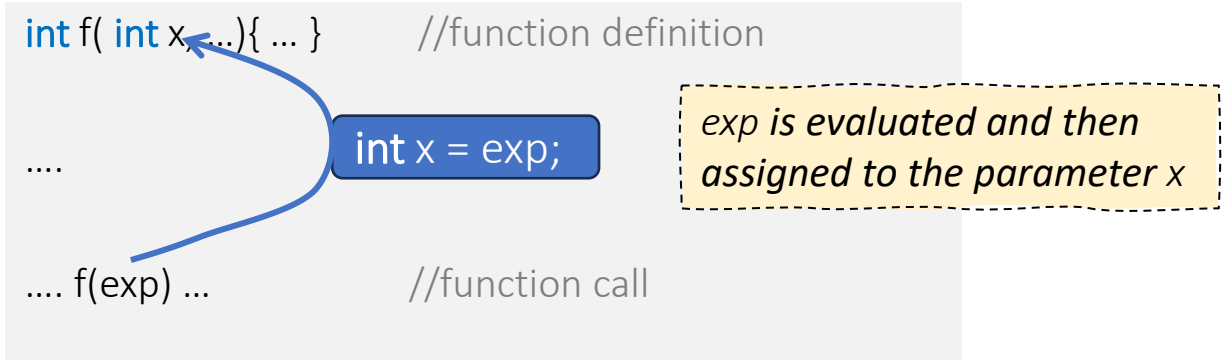
....

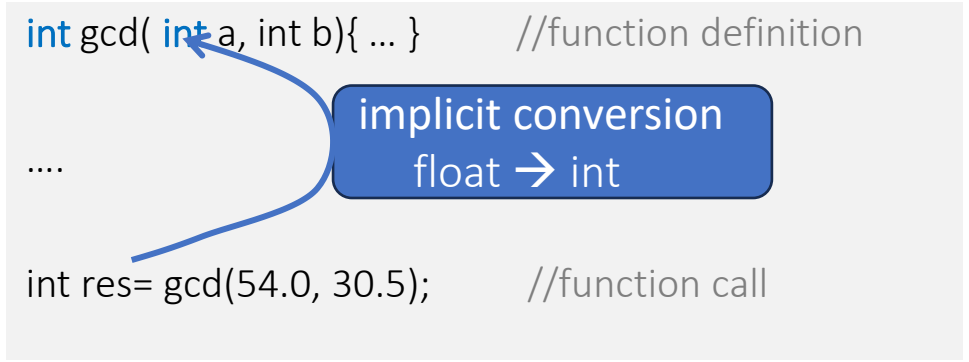
int res= gcd(54.0, 30.5);     //function call
```

implicit conversion
float → int

Implicit conversion of actual parameters

- The call (for value) is logically analog at the initialization of the formal parameters





- In the function call in which the actual parameters are of different but compatible types, with respect to the ones of the formal parameters, an implicit conversion is done (with all potential problems)
- The rules are like the ones of the initialization

Outline

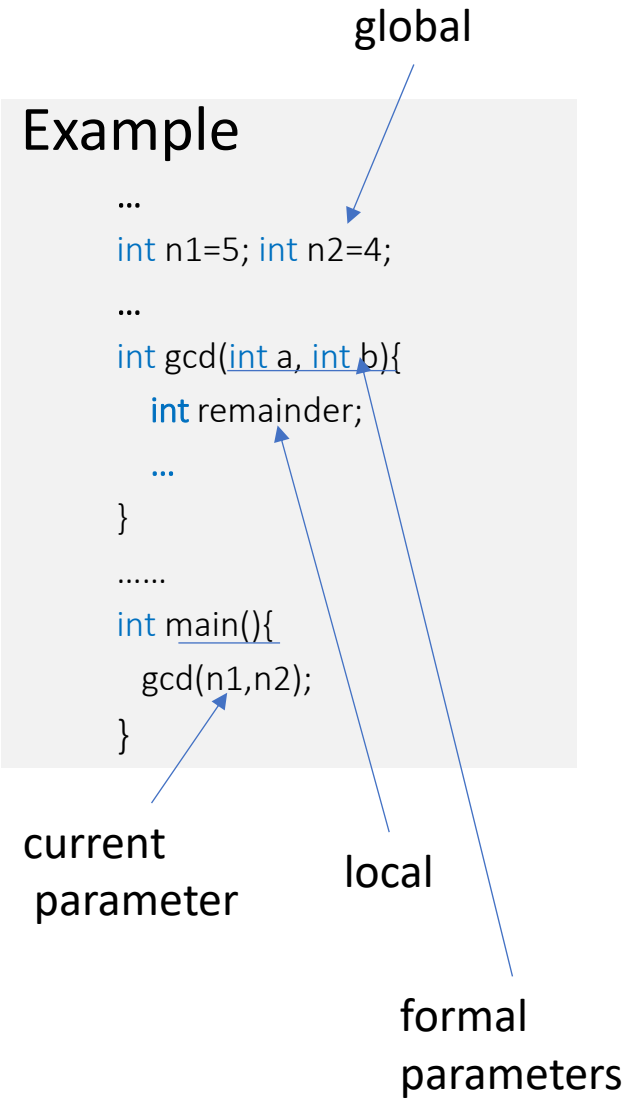
- Introduction (Functions, Library of functions, Procedures, …)
- Definition, Declaration and Call of Functions
- **Function parameters and local variables**
- Parameters passing
- Recursive functions

Parameters and local variables

- A formal parameter is a variable to which is associated the corresponding current parameter in each function call
  - [unless otherwise specified] the value of the current parameter is copied to the formal parameter (passing parameters **by value**)
- The variables declared within a function are called **local**
  - they belong only to the function in which they are declared
  - are visible only within the function
- The variables declared outside functions are called **global**
  - they are visible within each function (if they are not masked by local variables with the same name)

Example about the scope of local and global variables:

{ ../FUNCTIONS/scope.cc}

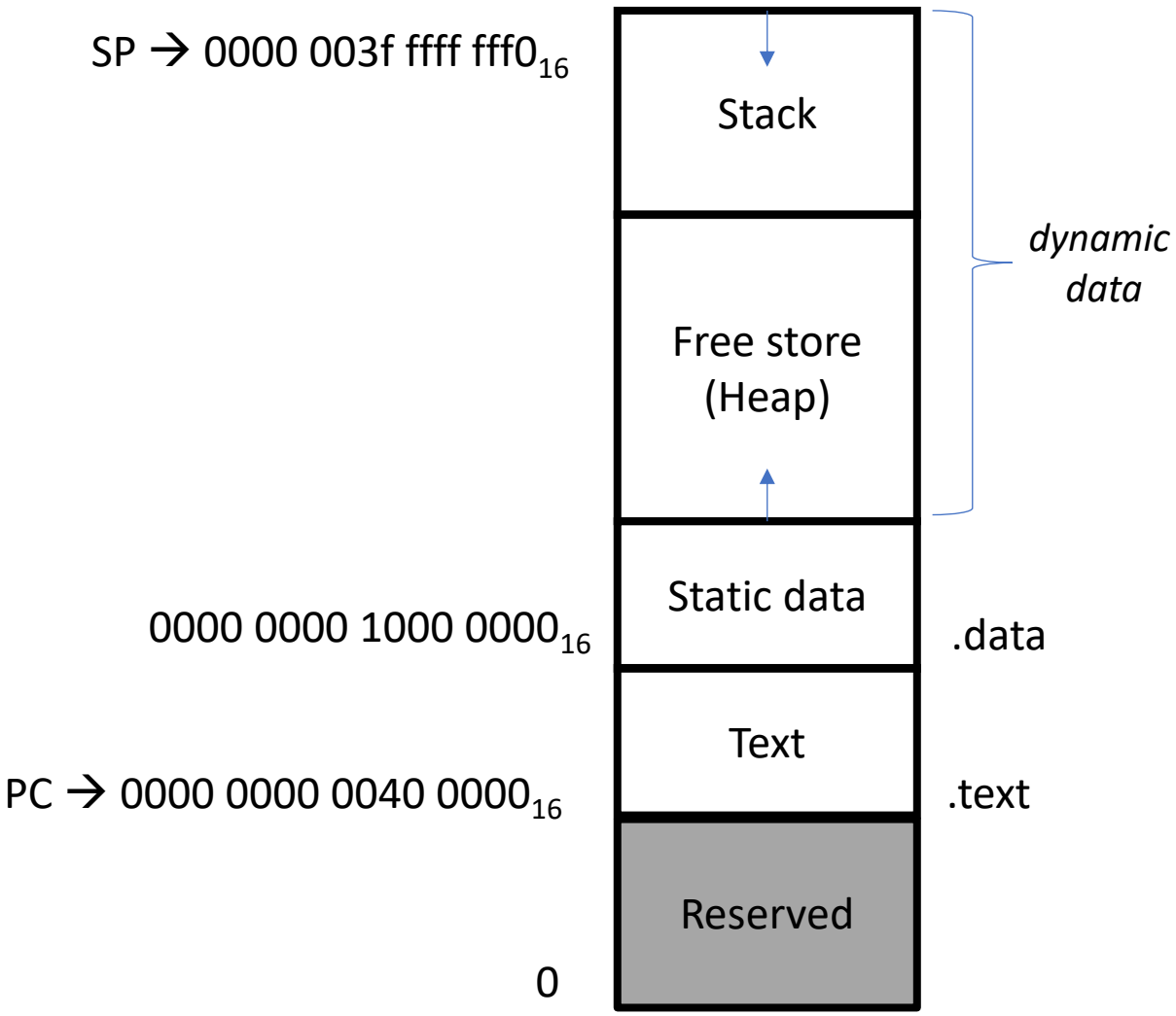


Duration of parameters and local variables

Formal parameters and local variables "exist" (they have memory space reserved to them) only during the execution of the respective function

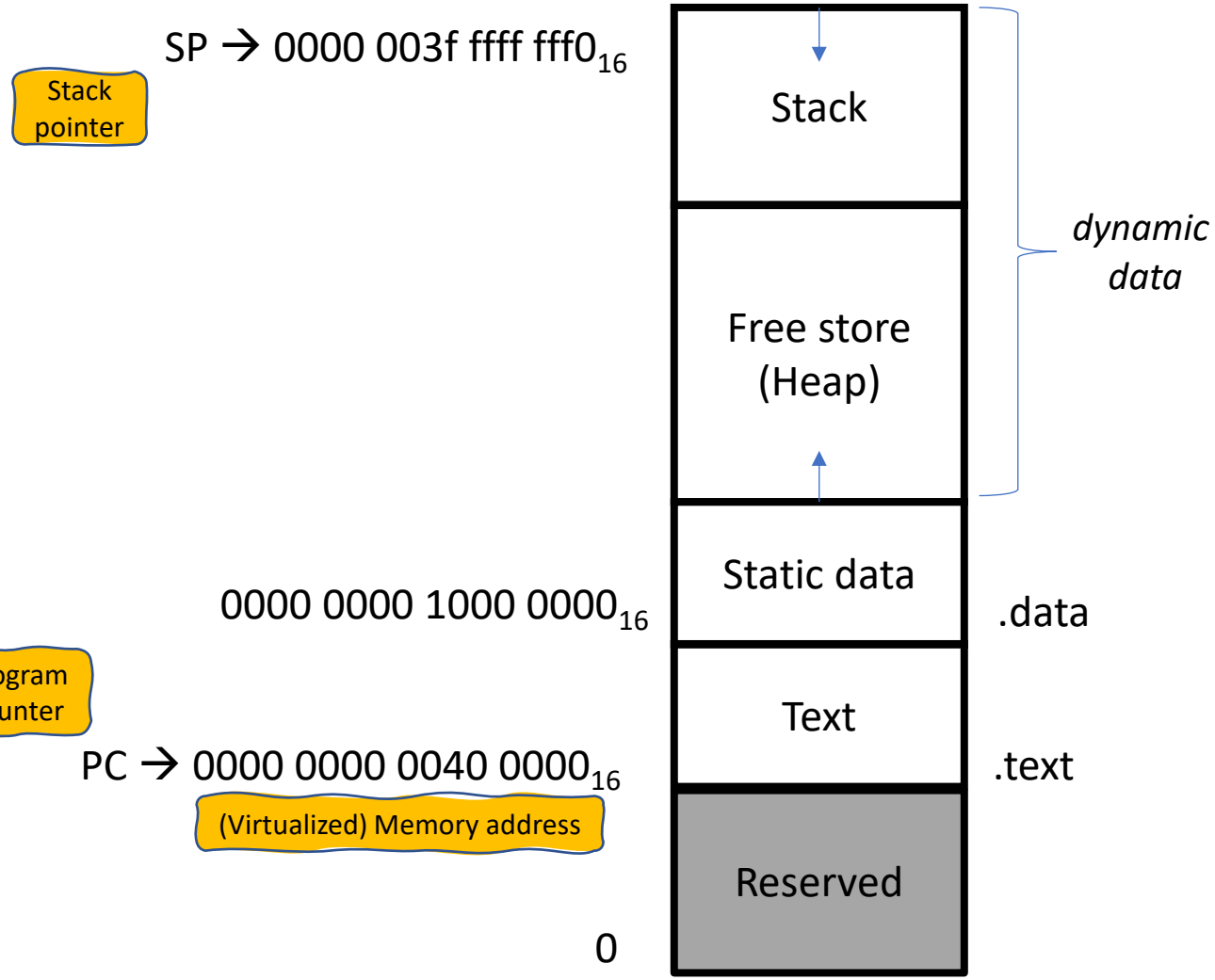
i. At the time of the call, a memory area is reserved for them
ii. They are used for the necessary processing (inside the function)
iii. At the end of the function, the memory occupied by them is made available

Structure of the memory of a program (1)



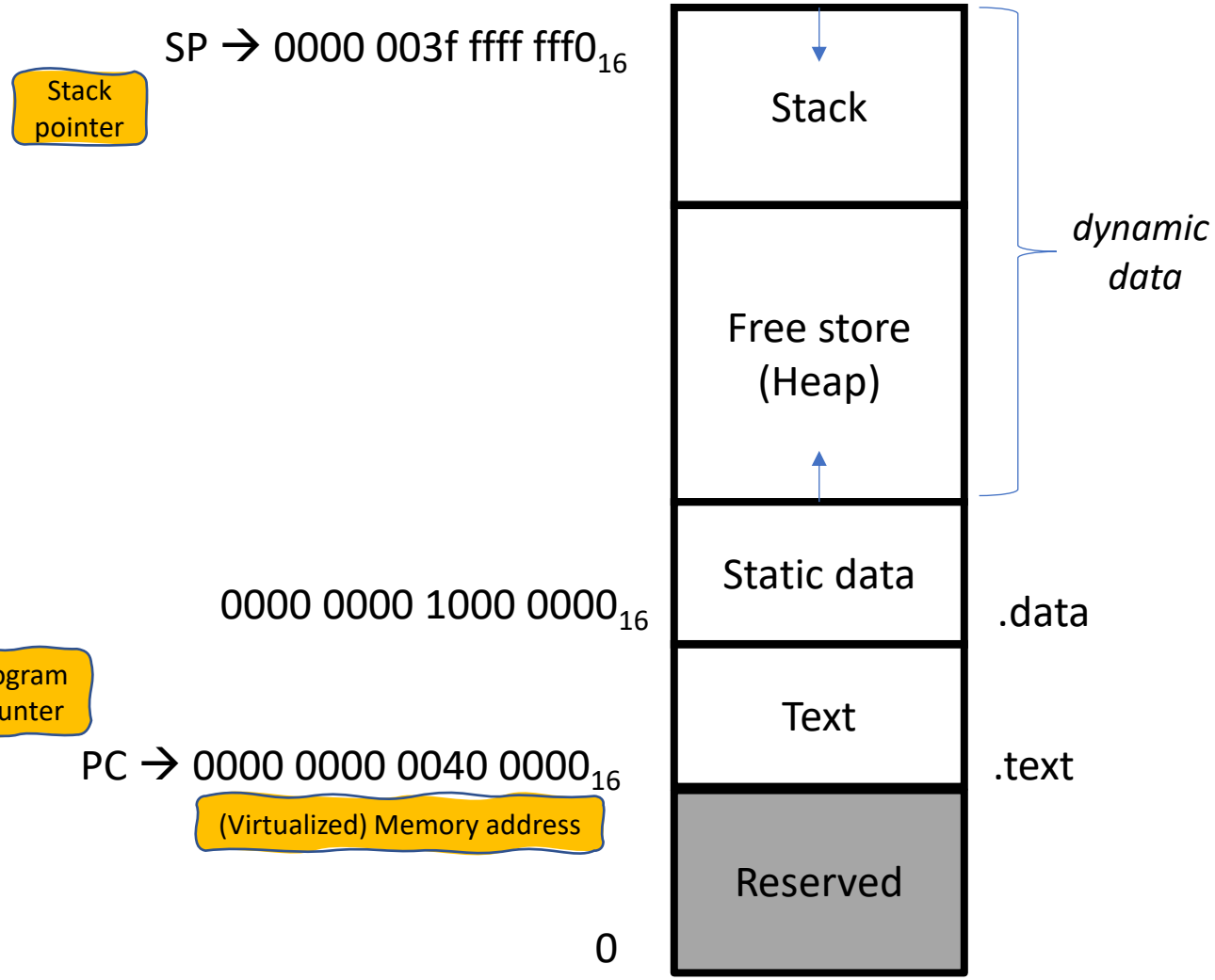
Different areas to allocate the different types of static and dynamic data and program information, e.g., the program code, global and local variables etc.

Structure of the memory of a program (2)



Reserved memory area. It contains machine instruction reserved for the Operating System (e.g., for booting the system and the program).

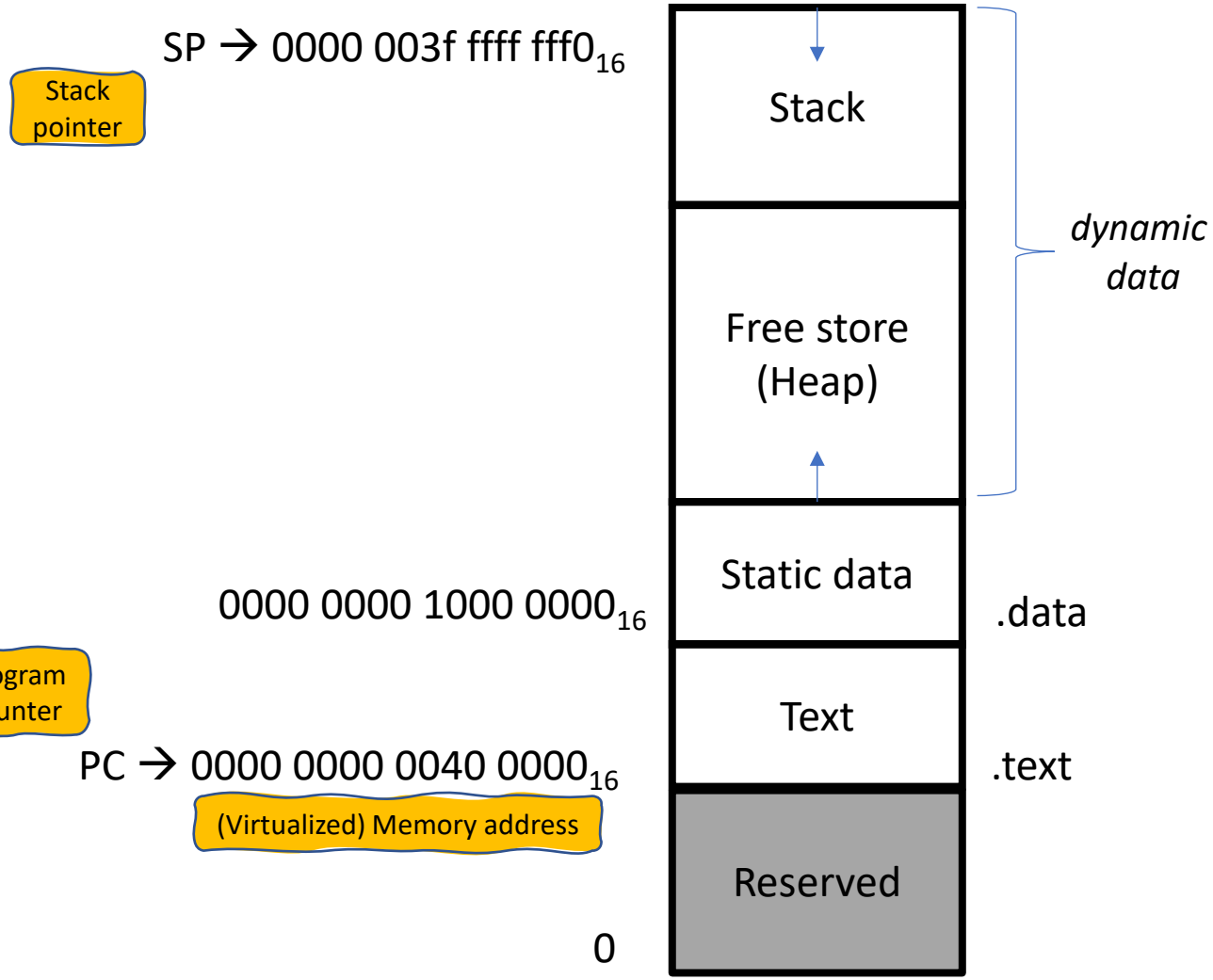
Structure of the memory of a program (2)



Text area. It contains assembly instruction of the program we are developing. This area is controlled by the compiler

Reserved memory area. It contains machine instruction reserved for the Operating System (e.g., for booting the system and the program).

Structure of the memory of a program (2)

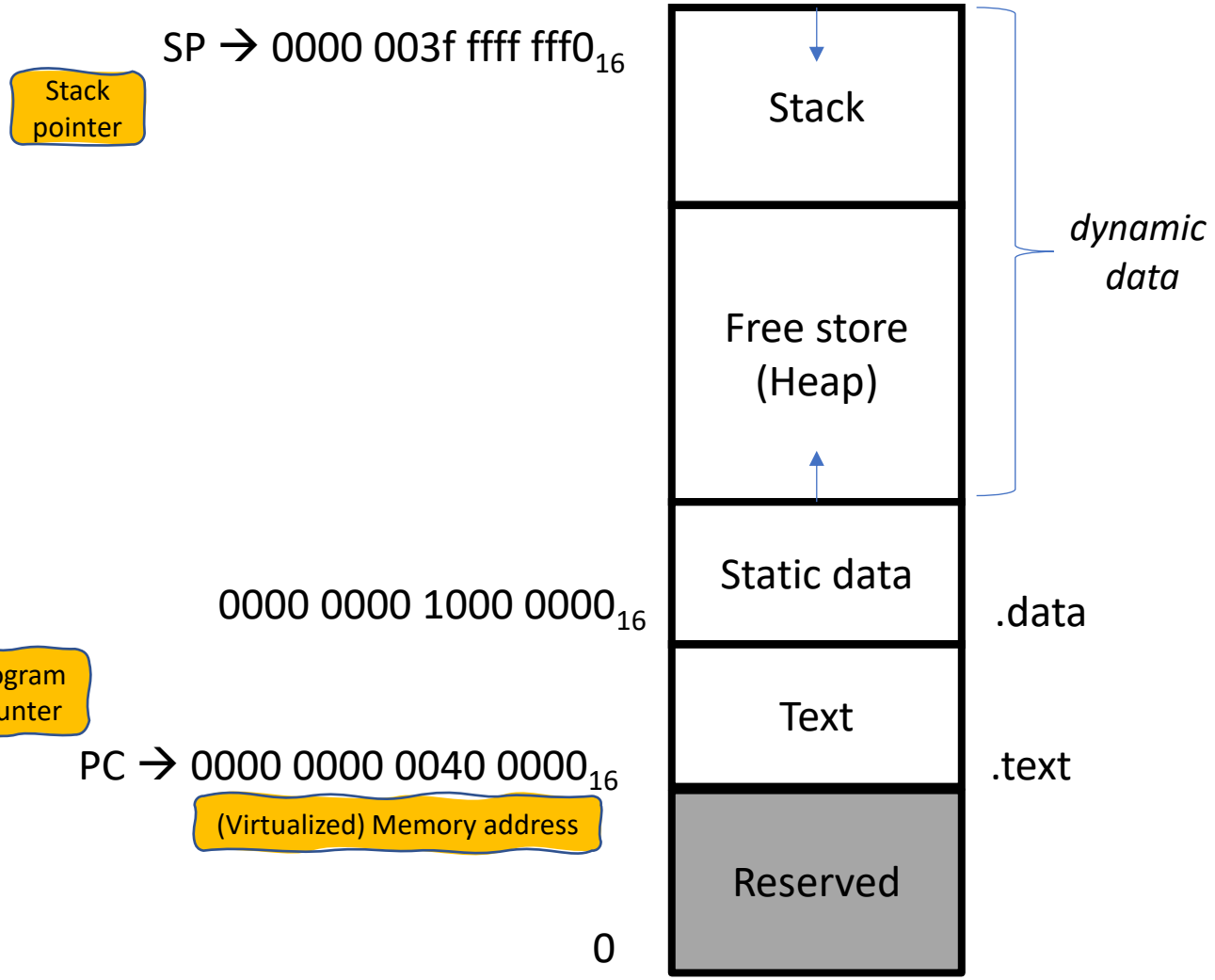


Static data. It contains constant, global variables and static variables declared in the program.

Text area. It contains assembly instruction of the program we are developing. This area is controlled by the compiler

Reserved memory area. It contains machine instruction reserved for the Operating System (e.g., for booting the system and the program).

Structure of the memory of a program (2)



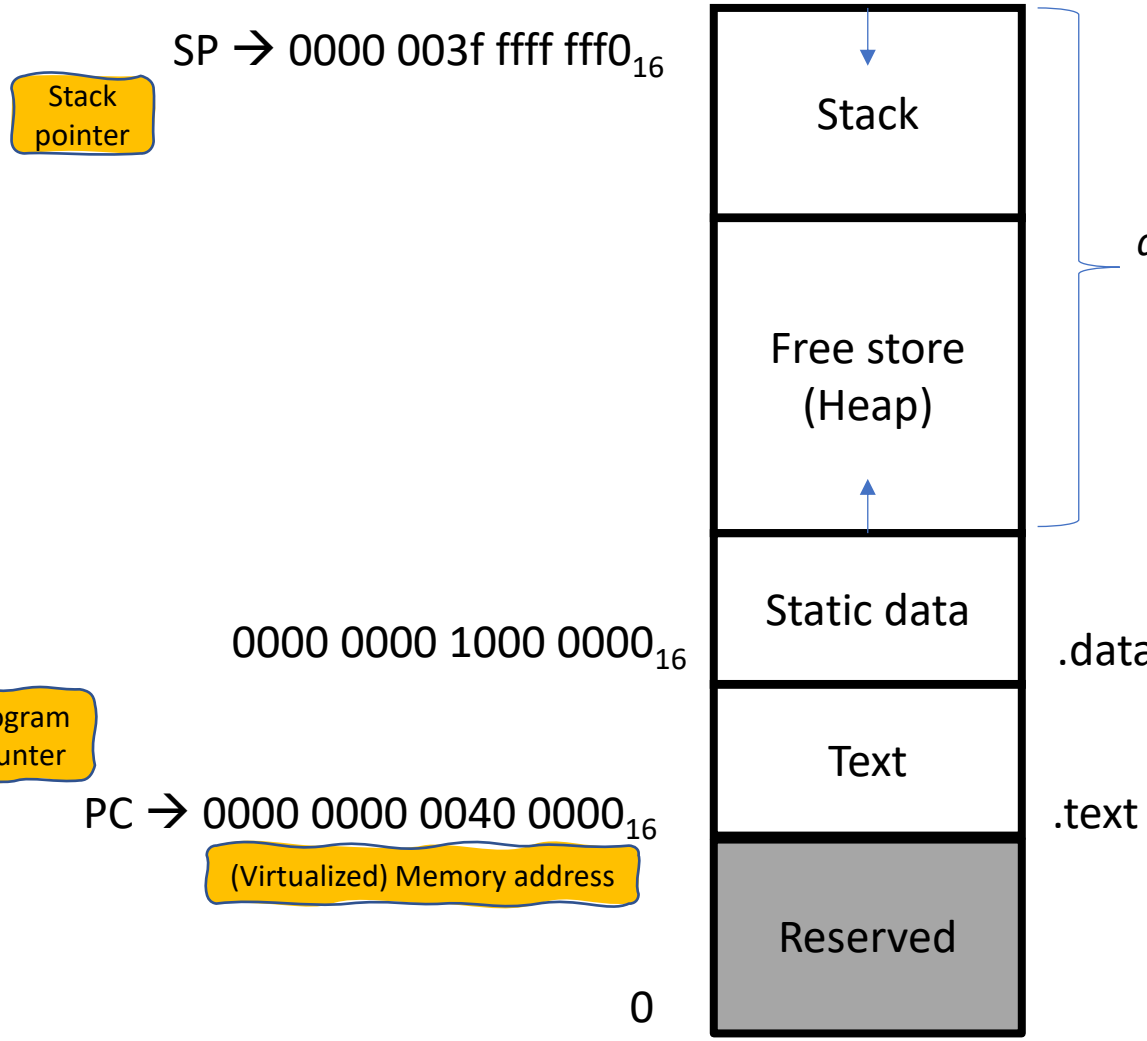
Heap. It is a dynamic area controlled by the developer (e.g., new, malloc, realloc, free). It contains dynamic information (for objects, functions and the dynamic allocation of the memory, i.e., dynamic data structure). E.g., if we declare variables (new) in a function, they will be allocated here. It is globally accessible (also outside the current scope) and used with more persistent data.

Static data. It contains constant, global variables and static variables declared in the program.

Text area. It contains assembly instruction of the program we are developing. This area is controlled by the compiler

Reserved memory area. It contains machine instruction reserved for the Operating System (e.g., for booting the system and the program).

Structure of the memory of a program (2)



Stack. The call stack keeps track of the active functions and handles allocation of all function parameters and local variables. It contains the program stack (e.g., function calls, reference to the used variables, function parameters). In Windows the stack size is fixed (4MB), in Linux is fixed but it can be changed programmatically.

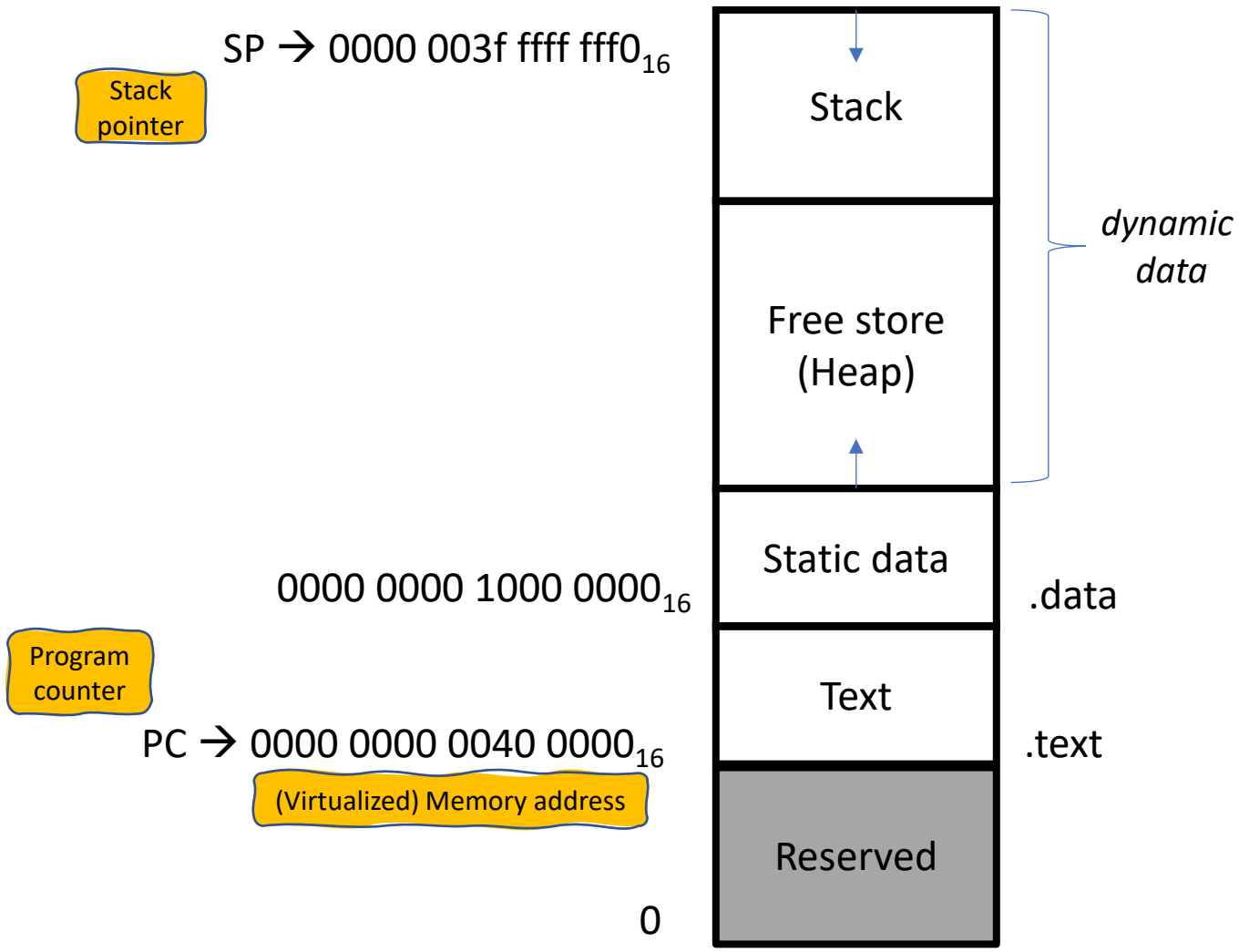
Heap. It is a dynamic area controlled by the developer (e.g., new, malloc, realloc, free). It contains dynamic information (for objects, functions and the dynamic allocation of the memory, i.e., dynamic data structure). E.g., if we declare variables (new) in a function, they will be allocated here. It is globally accessible (also outside the current scope) and used with more persistent data.

Static data. It contains constant, global variables and static variables declared in the program.

Text area. It contains assembly instruction of the program we are developing. This area is controlled by the compiler

Reserved memory area. It contains machine instruction reserved for the Operating System (e.g., for booting the system and the program).

Structure of the memory of a program (2)

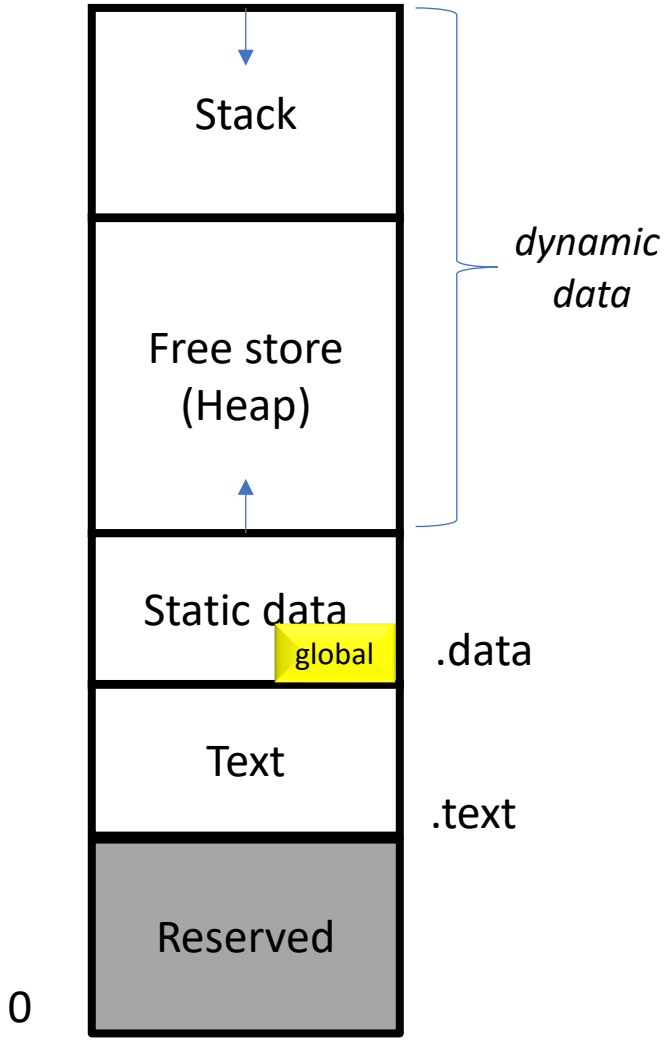


Stack. The call stack keeps track of the active functions and handles allocation of all function parameters and local variables. It contains the program stack (e.g., function calls, reference to the used variables, function parameters). In Windows the stack size is fixed (4MB), in Linux is fixed but it can be changed programmatically.

Stack Model: the memory is conceptually like to that of a "stack":

- when a function is called, the block of memory needed to hold its formal parameters and local variables is allocated "above" the one of the calling function
- when the function ends, that block is made available again
- "last in first out" (LIFO) management policy

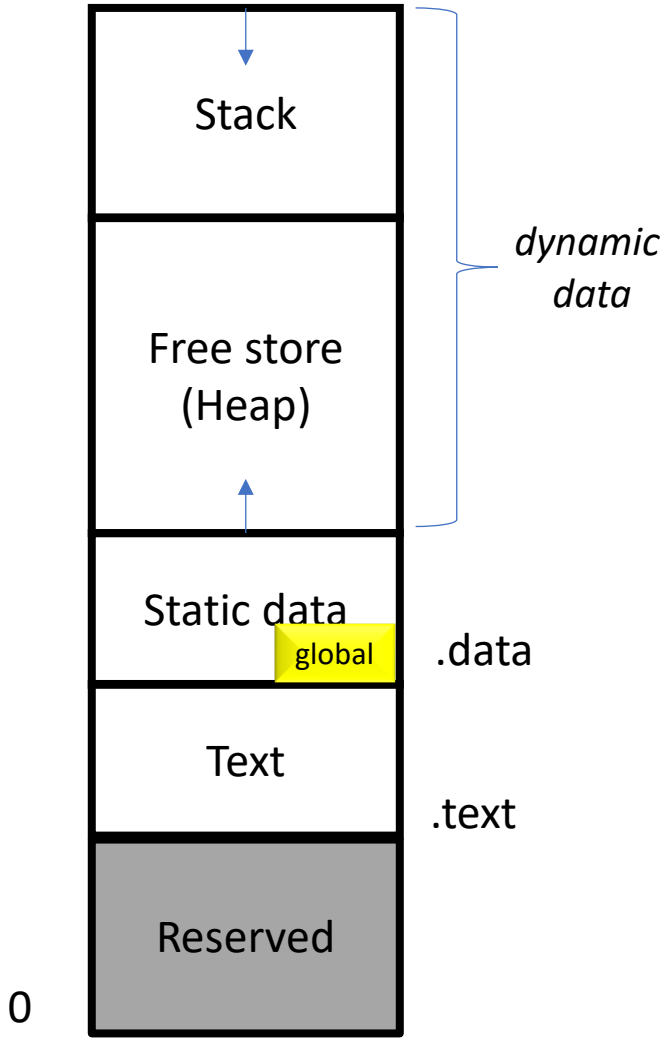
Structure of the memory of a program (3): example (1)




```
int global;

int main(void)
{
   return 0;
}
```

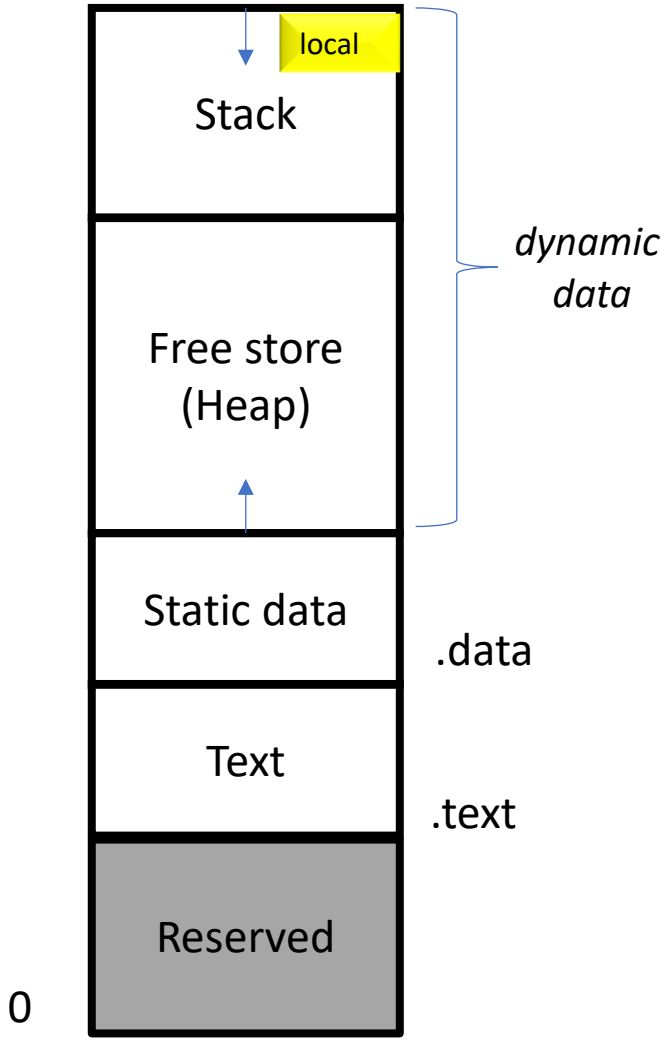
Structure of the memory of a program (3): example (2)




```
int global = 10;

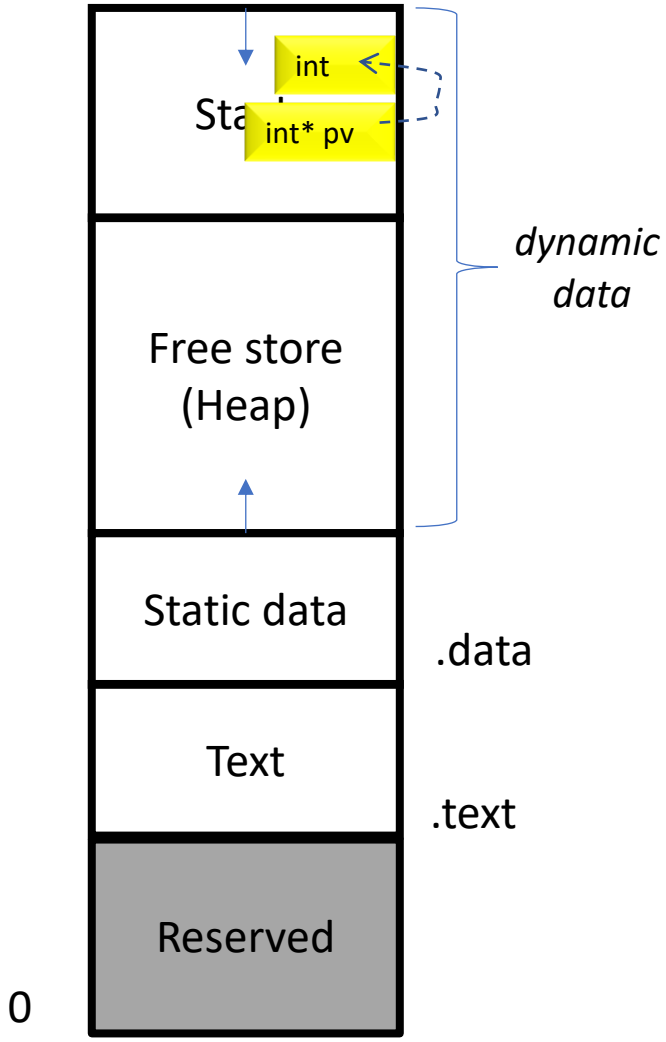
int main(void)
{
   return 0;
}
```

Structure of the memory of a program (3): example (3)




```
int main(void)
{
  int local =5;
   return 0;
}
```

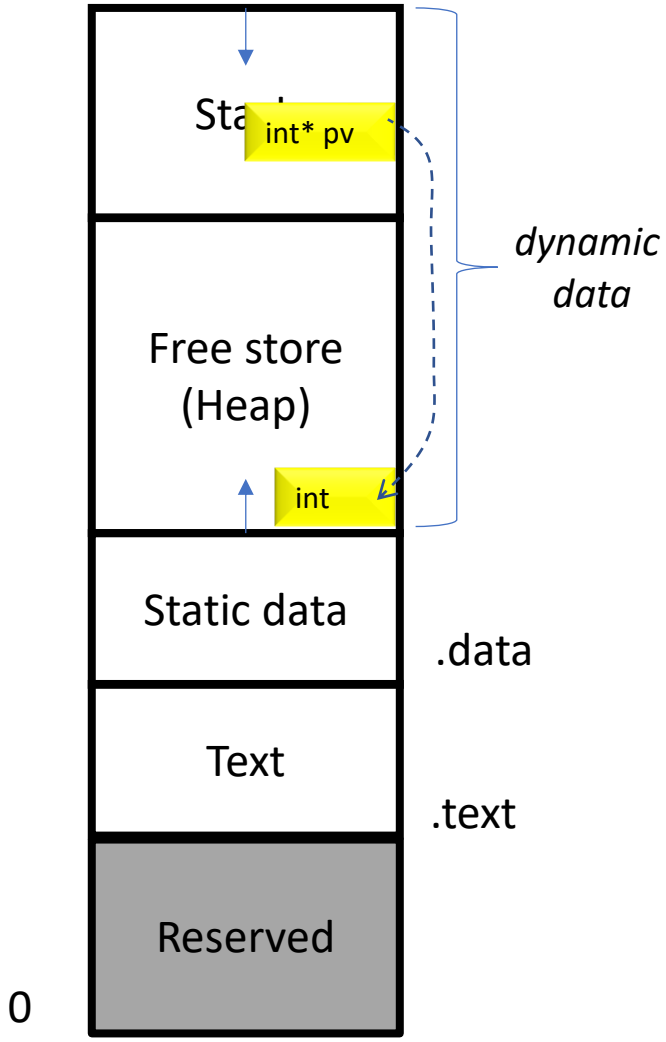
Structure of the memory of a program (3): example (4)




```
int main(void)
{
   int v= 100;
   int* pv = &v;
   return 0;
}
```

int* pv is the pointer

Structure of the memory of a program (3): example (5)

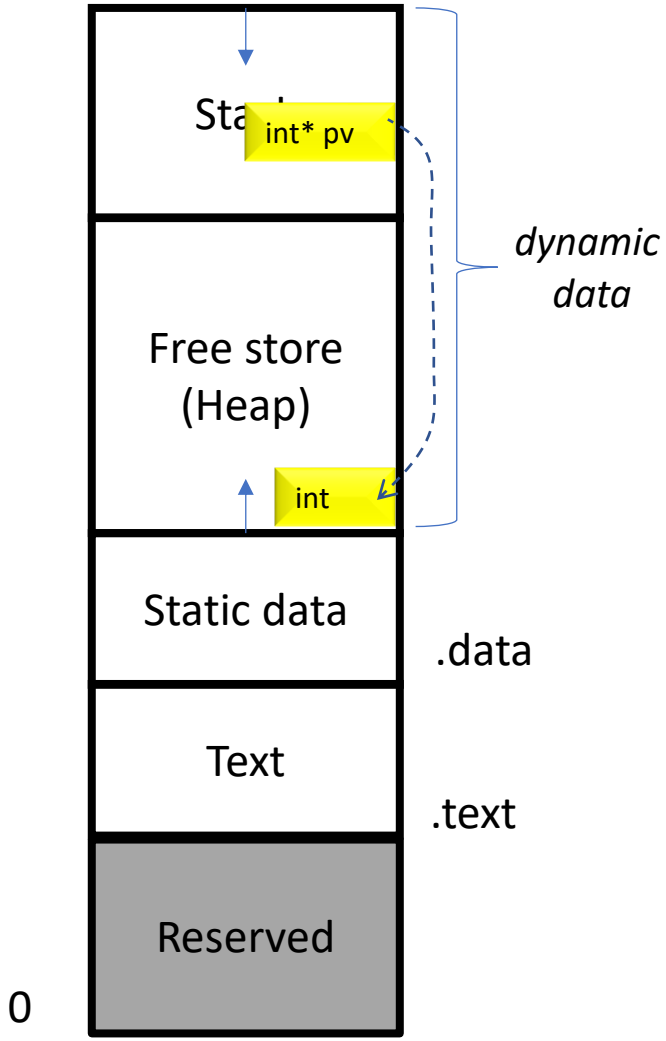


(later in the course)

```
int main(void)
{
   int* pv = new int[1];
   pv[0]=100;
   return 0;
}
```

int* pv is a pointer

Structure of the memory of a program (3): example (6)



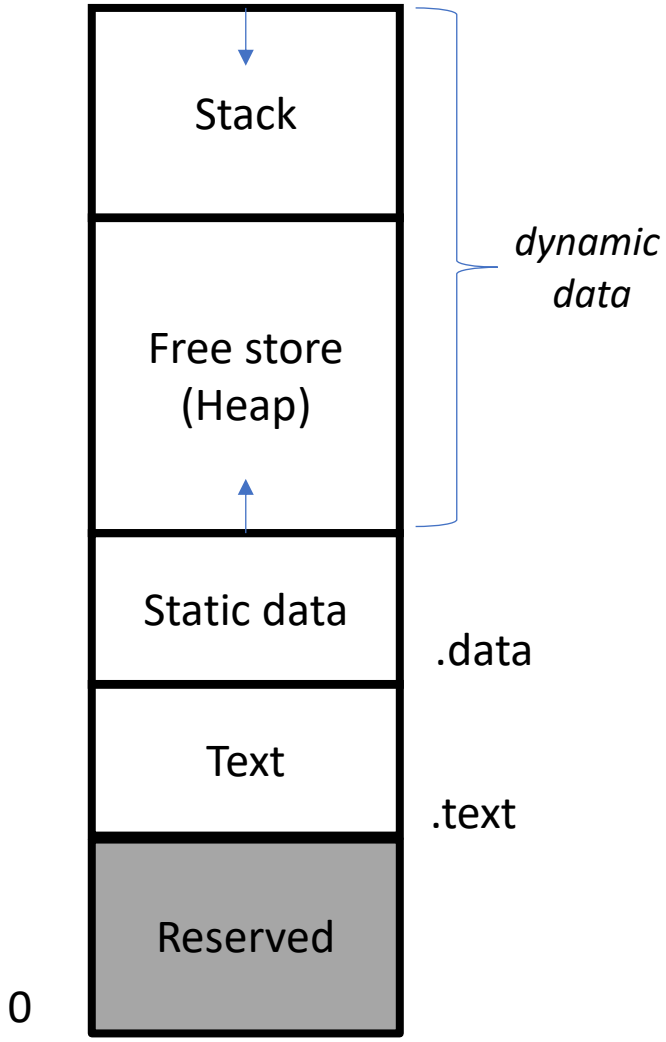
(later in the course)

```
#include <cstdlib>
int main(void)
{
   int* pv = (int*)malloc(sizeof(int));
   *pv = 20;
   return 0;
}
```

int* pv is a pointer

malloc is not part of the topics of the course

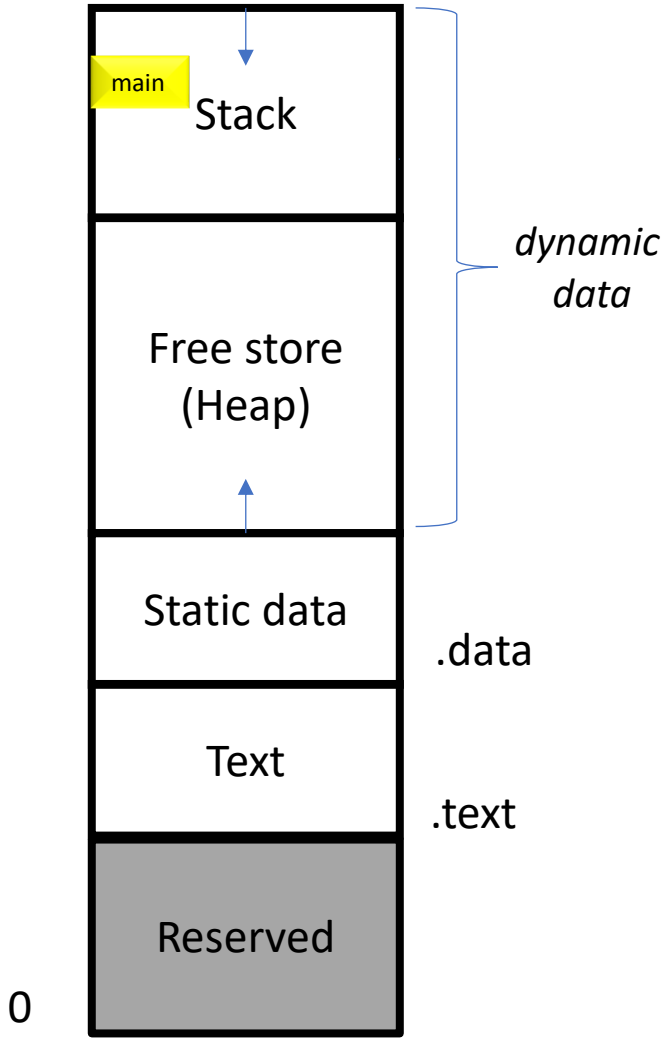
Structure of the memory of a program (3): example (4)




```
int funct(){
  int a = 100
  return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

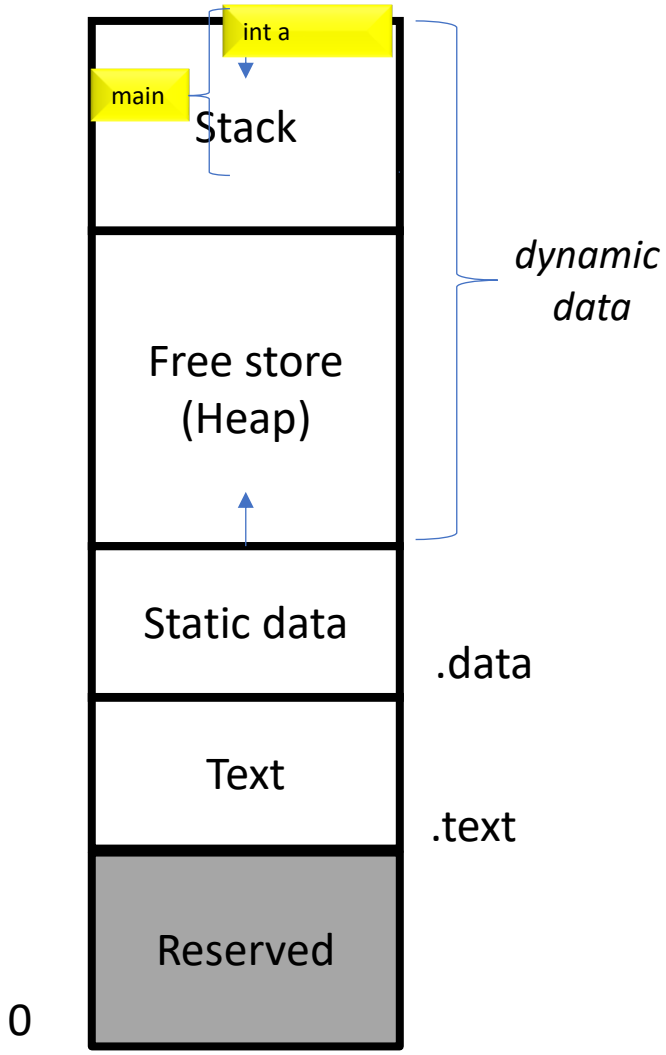
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

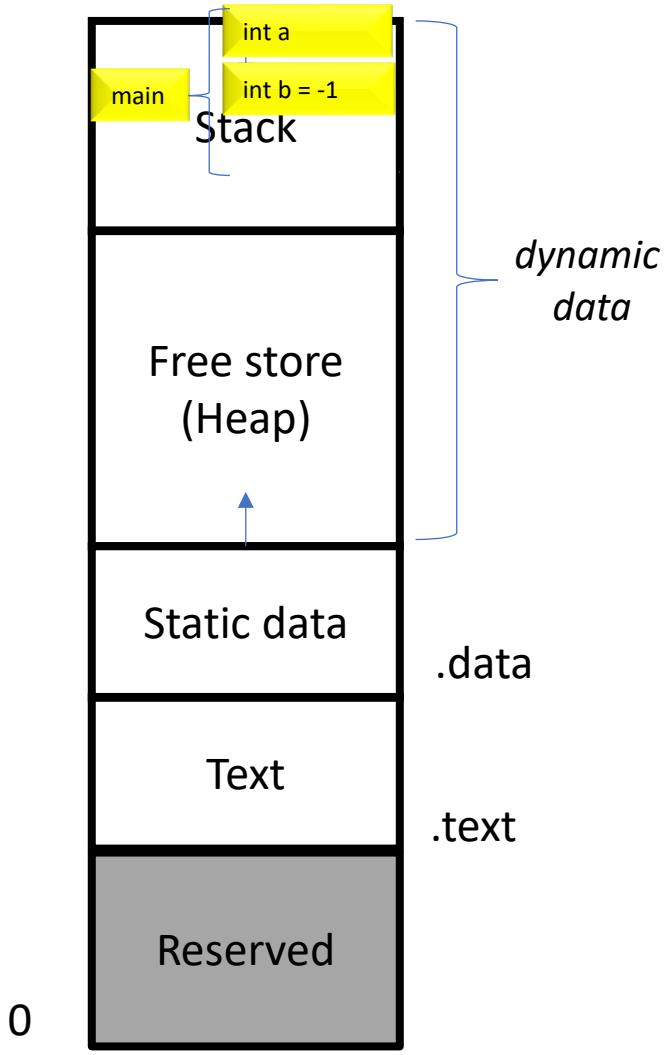
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

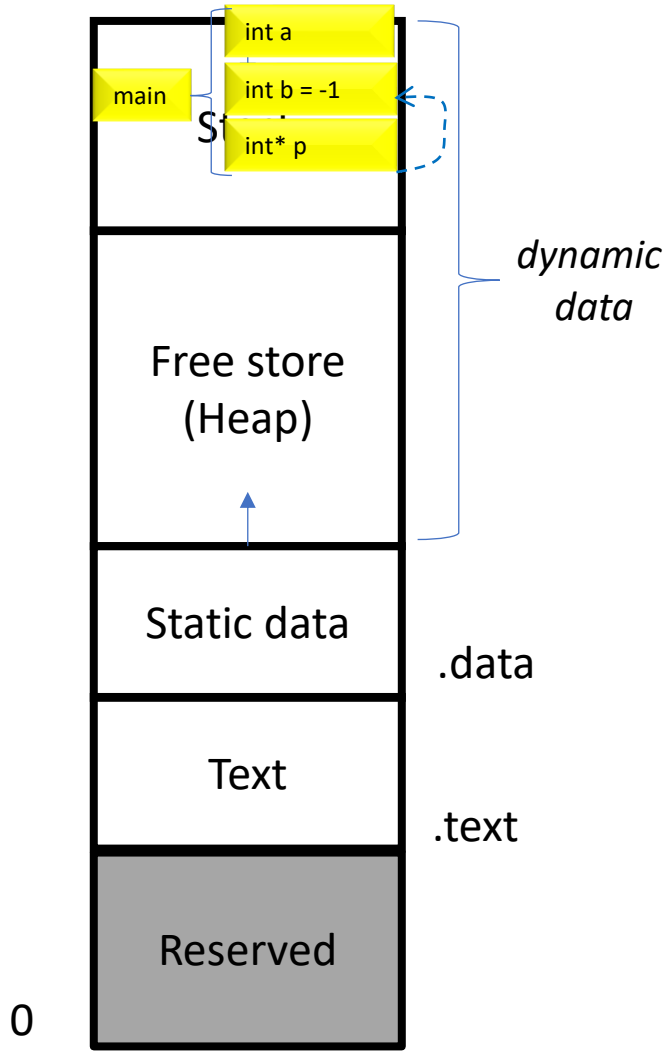
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

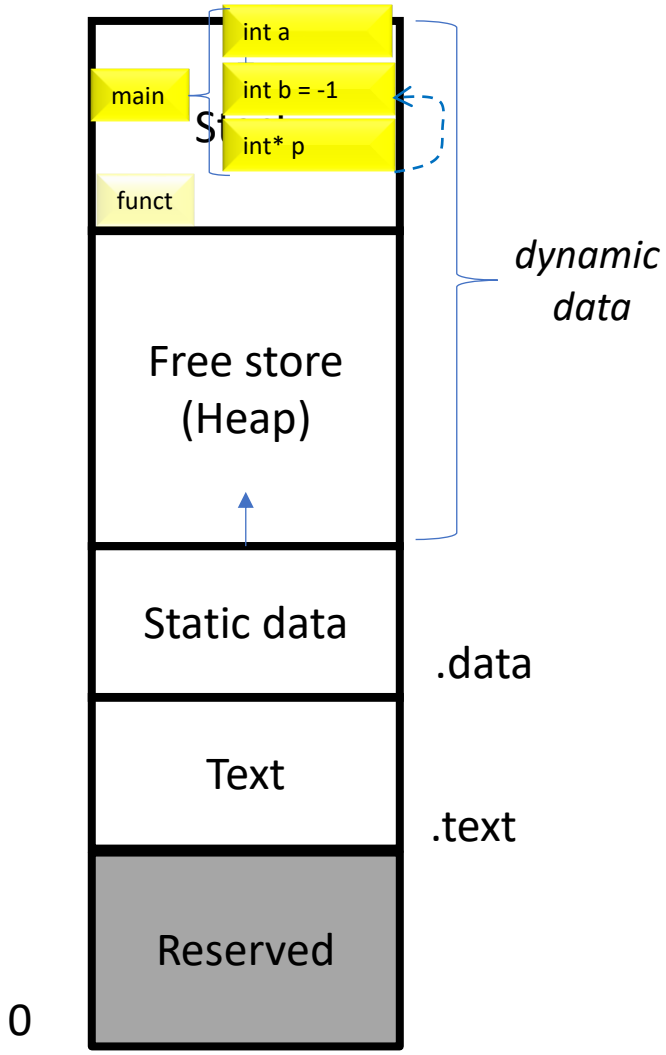
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

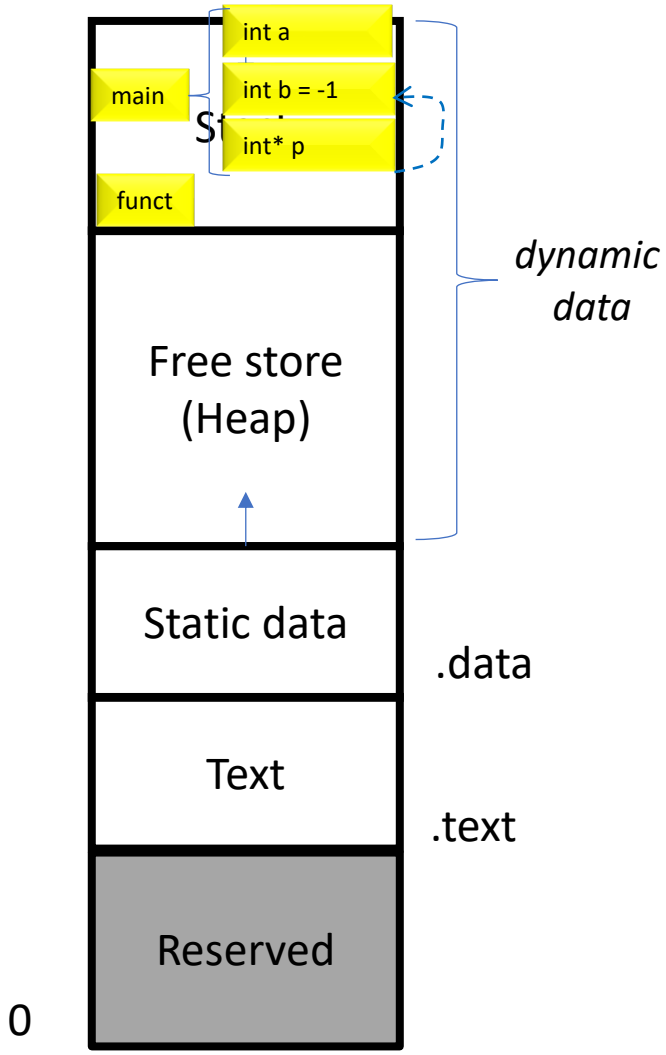
Structure of the memory of a program (3): example (4)




```
int funct(){
  int a = 100
  return a;
}

int main(){
  int a;
  int b=-1;
  int *p = &b;
  int r = funct();
  return 0;
}
```

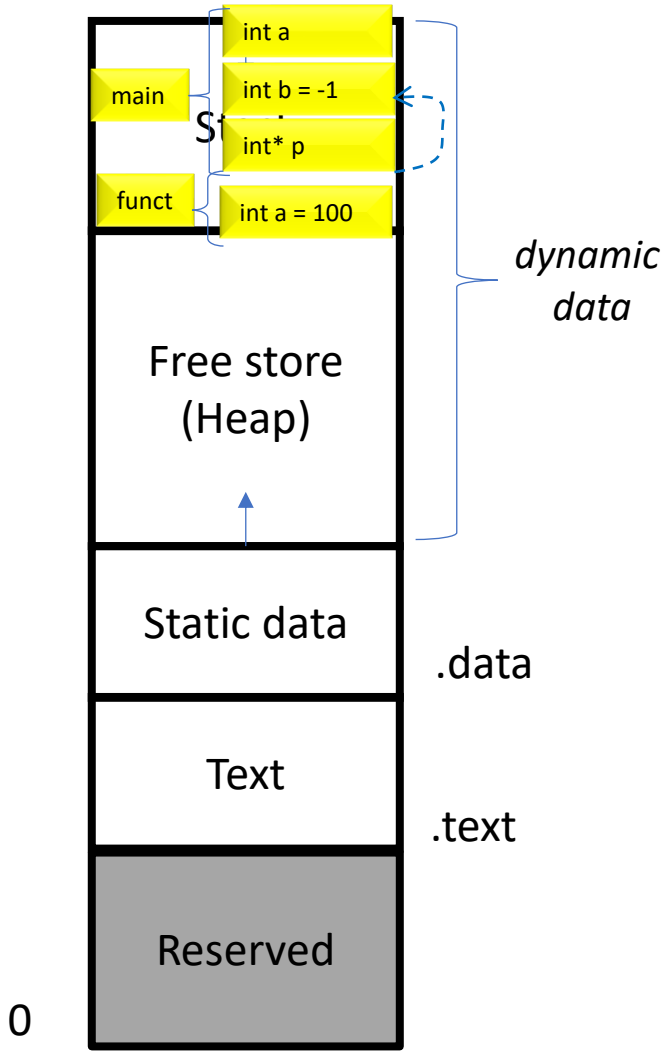
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

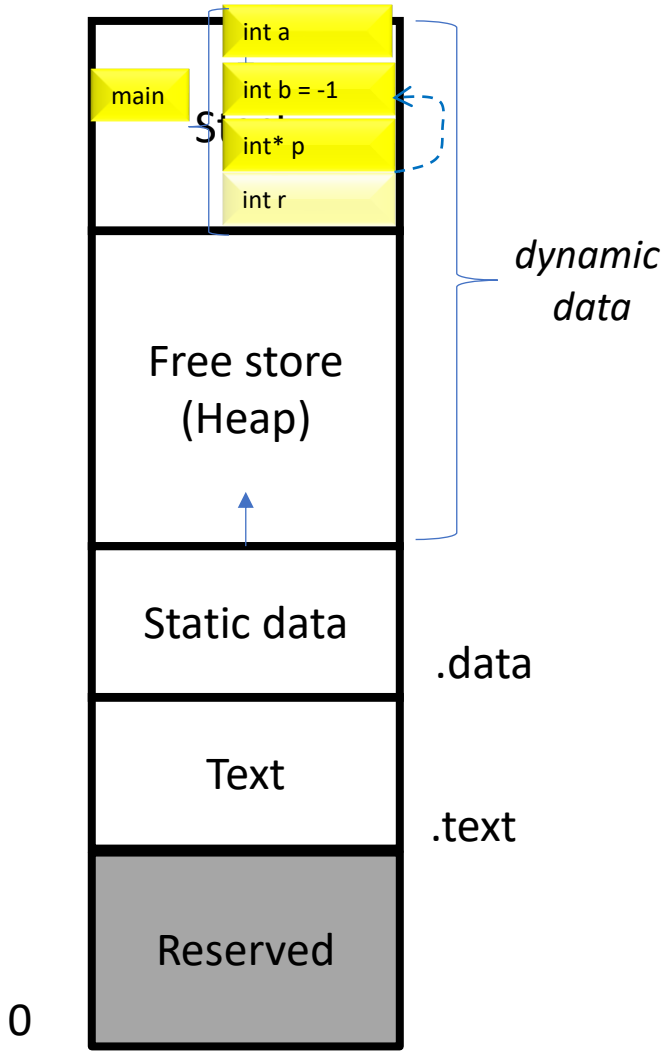
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

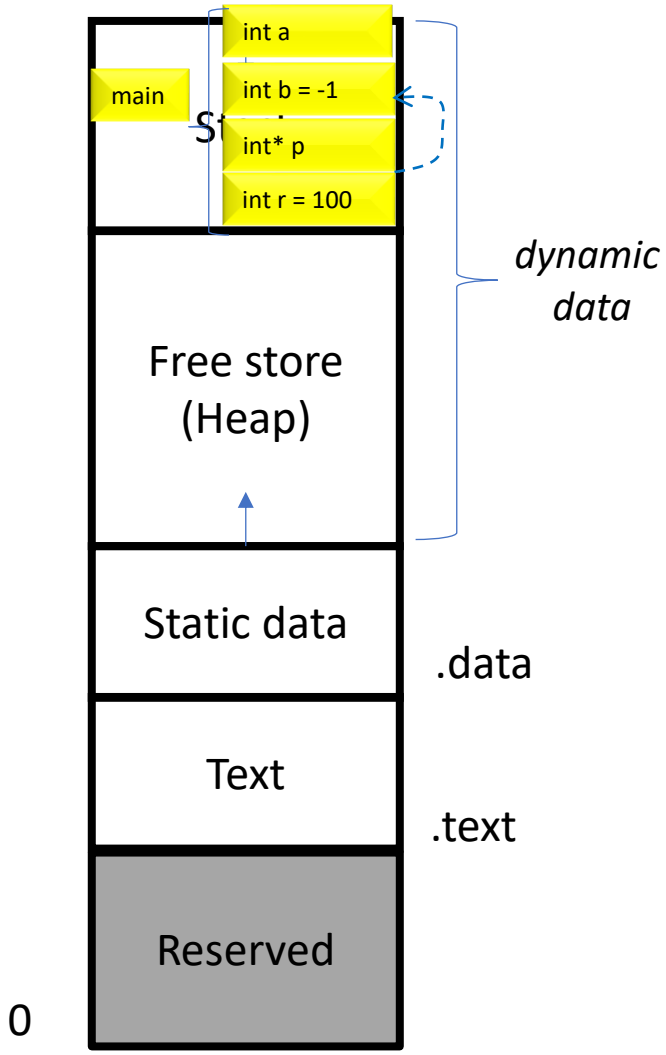
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

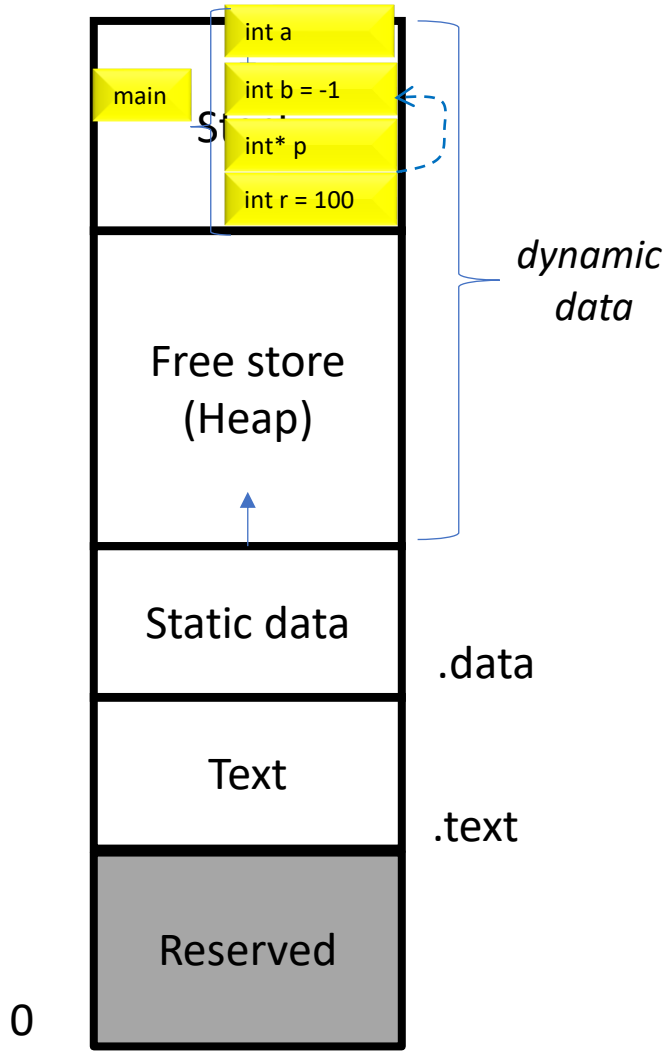
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

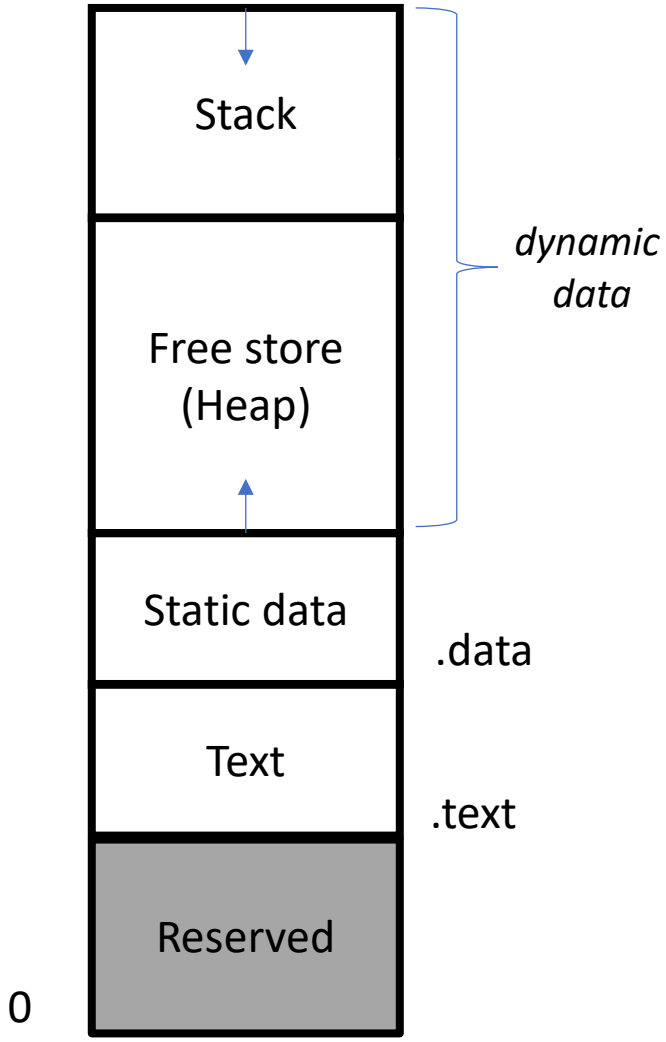
Structure of the memory of a program (3): example (4)




```
int funct(){
 int a = 100
 return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

Structure of the memory of a program (3): example (4)




```
int funct(){
  int a = 100
  return a;
}

int main(){
 int a;
 int b=-1;
 int *p = &b;
 int r = funct();
 return 0;
}
```

Examples

Example of functions that call function [D]:

{ ../FUNCTIONS/comb.cc}

Example with declarations (headers):

{ ../FUNCTIONS/comb2.cc}

Example of tracing the address of variables and parameters:

{ ../FUNCTIONS/comb2_track.cc}

Example with nested functions [D]

{ ../FUNCTIONS/**mymax**.cc}

Example of tracing the addresses (stack):

{ ../FUNCTIONS/**mymax_track**.cc}